

★ Member-only story

AI + Gemma 4 + Programming + Python +

I Tested Gemma 4 Against Claude Opus 5 and GPT 5.5 on Real Python Projects. The Results Surprised Me.



inprogrammer

Following ▾

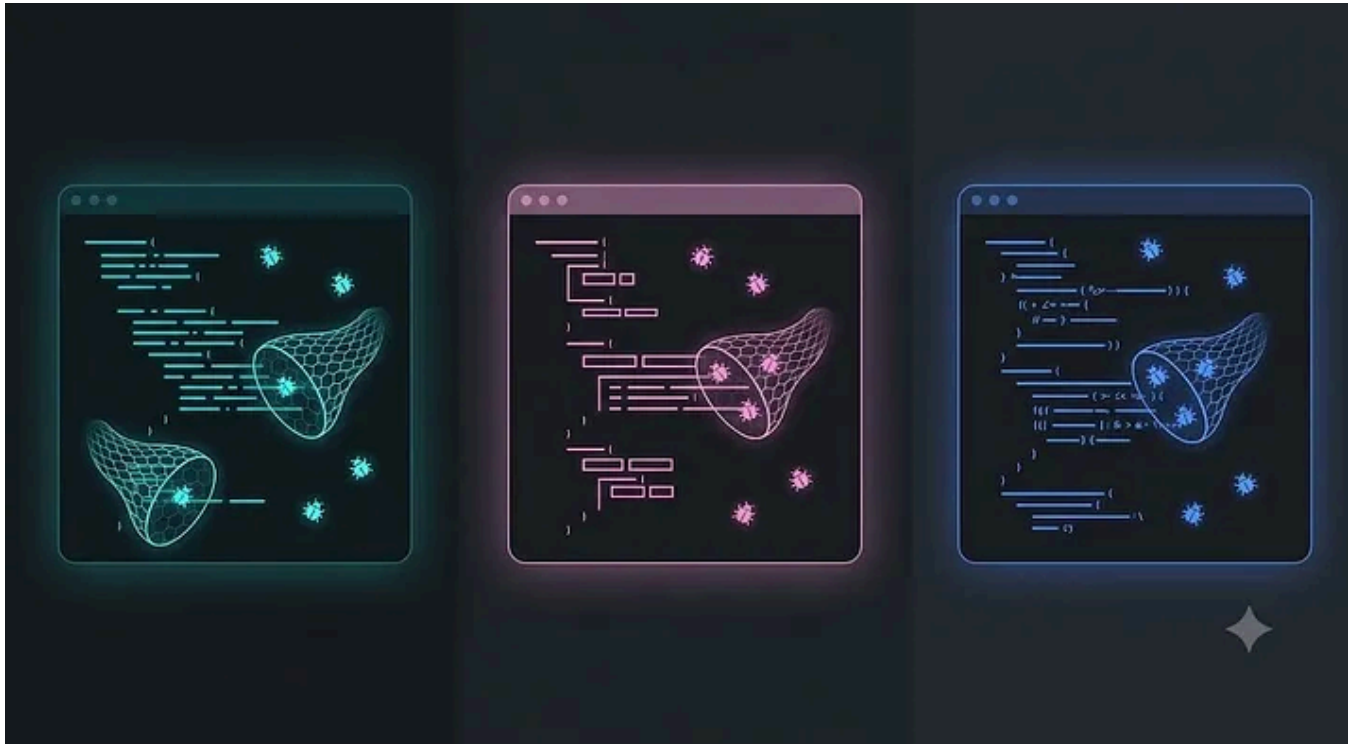
6 min read · 9 hours ago



1



I almost did not write this piece.



gemini (nano banana)

Here,s the Friends [link](#)

Every week there is a new model claiming to be the best coding assistant on the planet, and every week the benchmark charts look identical: a bar graph, a green arrow, a headline that says “state of the art.” I stopped trusting those charts a long time ago. So instead of reading another leaderboard, I opened



The contenders were Gemma 4, Google DeepMind's open weight model that you can run on your own machine, Claude Opus 5, Anthropic's new everyday workhorse model released this July, and GPT 5.5, OpenAI's flagship released back in April. One is free and local. Two are paid and cloud hosted. I wanted to know if the price gap actually buys you anything when the work is real instead of synthetic.

A quick note before anyone asks: yes, GPT 5.6 shipped publicly in July. I stuck with GPT 5.5 here because it is still the version most teams have actually rolled out in production, and I wanted a fair fight between three models that developers are using today, not the newest release by a few weeks. A GPT 5.6 rematch is coming as a follow up post, I will link it right here once it is published.

How I set this up

So this is reproducible if you want to run your own version:

- Gemma 4 31B Dense, Q6 quantization, served locally through Ollama on a workstation with an RTX 4090 and 64GB of system RAM, 128K context window.
- Claude Opus 5, default effort setting, accessed through the Claude API, 200K context window.

- GPT 5.5 Thinking, accessed through the OpenAI API, default reasoning effort.
- Each model connected to the same repo through a simple tool calling harness that could read files, run the test suite, and apply patches. No custom prompt engineering beyond a short system message describing the repo and the task.
- Every task ran three times per model and I report the median.

None of the three tasks came close to filling even the smaller 128K window, so the extra headroom on Claude Opus 5 never actually came into play here. It would matter more on a task that involved reasoning across a much larger codebase.

The three tests

I kept the tasks close to what actually shows up in a backend engineer's day, not puzzle box trivia.

Test one, the async bug hunt. A FastAPI service with a Celery worker that occasionally dropped tasks under load. No hints, just the traceback and the repo.

Test two, the messy data pipeline. A script that reads a large CSV, does some pandas transforms, and chokes on memory when the file grows past a few gigabytes. The ask was simple: make it not fall over, without rewriting the whole thing in a different language.

Test three, the vague feature ticket. “Users should be able to filter their orders by date range, and it should be fast.” No schema, no acceptance criteria, just vibes.

SCORECARD DEVELOPER EXPERIENCE BENCHMARKS			
Task	 Gemma 4	Claude Opus 5	 GPT 5.5
Async bug hunt	 Partial, fixed after one nudge	 Pass, first attempt	 Pass, first attempt
Data pipeline	 Pass, but over engineered	 Pass, first attempt	 Pass, first attempt
Vague ticket	 Pass, no questions asked	 Pass, asked two questions...	 Pass, documented assumption...
 = Pass,  = Partial Pass			

gemini (nano banana)

METRICS			
Metric	 Gemma 4	 Claude Opus 5	 GPT 5.5
Median time to working fix	 4m 40s	 2m 55s	 3m 20s
Approx tokens used per task	 ~9,000 (local, no cost)	 ~6,500	 ~11,000
Caught issue I did not ask about	 No	 Yes, dtype upcasting	 No

Data-driven comparative analysis, Q3 2024

gemini (nano banana)

What happened with the async bug

The bug was a shared mutable default argument on the retry handler, combined with a race condition once two workers hit the same task at once.

python

```
# Before, the actual bug
def enqueue_retry(task_id, seen_ids=[]):
    seen_ids.append(task_id)
    if seen_ids.count(task_id) > MAX_RETRIES:
        raise TaskFailed(task_id)
    retry_task.delay(task_id)
```

Claude Opus 5 found both problems on its first pass and rewrote it like this, then added a test that reproduced the original failure before I even asked for one:

python

```
# After, Claude Opus 5's fix
def enqueue_retry(task_id, seen_ids=None):
    seen_ids = seen_ids or set()
    seen_ids.add(task_id)
    with retry_lock:
        if retry_counts[task_id] > MAX_RETRIES:
            raise TaskFailed(task_id)
        retry_counts[task_id] += 1
    retry_task.delay(task_id)
```

It was also the slowest of the three to produce this particular fix, about 15 seconds behind GPT 5.5, likely the tradeoff for writing and running that extra reproduction test before committing to the patch. Worth it in my opinion, but it is the one place Opus 5 was not simply faster and better across the board.

GPT 5.5 landed on an equivalent fix a little slower than Gemma 4's first token but faster than Opus 5 overall, with heavier inline commentary explaining why mutable defaults are dangerous in Python. Useful if you are handing this to a junior engineer, noisy if you already know the pattern.

Gemma 4 caught the mutable default immediately but missed the race condition on its first attempt, patching only the append bug. It caught the lock issue once I pointed at the failing concurrency test. Given this ran locally for free, that is still a solid result.

What happened with the data pipeline

The core issue was in a merge step that quietly changed the dtype of the join key:

```
python
```

```
# Before, the actual bug
merged = orders_df.merge(customers_df, on="order_id", how="left")
# order_id was int32 in orders_df, but float64 in customers_df
# after a bad CSV import, silently upcasting the merged column
```

That silent upcast was inflating memory well beyond what the chunking fix alone would solve.

Claude Opus 5 rewrote the read into chunks and flagged the dtype issue unprompted:

python

```
for chunk in pd.read_csv(path, chunksize=200_000, dtype={"order_id": "int32"}):
    chunk["order_id"] = chunk["order_id"].astype("int32") # prevents upcast dur
    process(chunk)
```

GPT 5.5 produced a correct chunked version but did not touch the dtype problem until I asked why memory was still high after the fix.

Gemma 4 proposed dropping pandas for a different library entirely, which technically solves memory pressure but is a much bigger change than the ticket called for.

What happened with the vague ticket

All three produced working filter logic. Claude Opus 5 asked whether “date range” meant order creation or delivery date, and whether “fast” meant adding an index. GPT 5.5 picked reasonable defaults and documented them in comments. Gemma 4 implemented the most common interpretation immediately, no questions asked, fastest of the three but with the most assumption risk.

Cost per task

Rough cost using default API pricing, median of three runs, includes the follow up nudge where one was needed:

MODEL COST COMPARISON (PER TASK)

 MODEL	 COST PER TASK (APPROX)	 NOTES
 Gemma 4 31B	Effectively \$0 in API fees	 Electricity plus the GPU you already own
 Claude Opus 5	~\$0.04 to \$0.09 per task	 \$5 / \$25 per million input/output tokens
 GPT 5.5	~\$0.06 to \$0.12 per task	 Slightly more verbose output on average

gemini (nano banana)

None of these numbers will bankrupt anyone at this scale, but they compound fast across a team running hundreds of tasks a day, which is exactly where Gemma 4's zero marginal cost starts to matter, even if the real cost shows up as your own hardware and time instead of a bill.

The honest verdict

Claude Opus 5 was the most consistently careful of the three, verified its own work without being asked, and matched Anthropic's own framing of it as a lower cost model that still catches things a careless pass would miss. It was also the slowest to land the async fix of the three, a small price for the extra verification step.

GPT 5.5 remains dependable and easy to hand to less experienced teammates because of how much it explains inline, it just occasionally needs a nudge to go deeper.

Gemma 4 is the real story. Running for free on hardware I already owned, it got two out of three tasks essentially right on the first attempt, and the one place it fell short was recoverable with a single follow up message.

What I am actually doing

I am moving my day to day debugging work to Claude Opus 5, keeping GPT 5.5 as the backup when a teammate needs more explanation in the diff, and running Gemma 4 locally for anything involving code I am not allowed to send to a third party API.

If you have run your own version of this test, especially against GPT 5.6, I want to hear what you found. Drop it in the comments, I will fold the most

interesting results into the follow up post.

AI +

Gemma 4 +

Programming +

Python +



Written by inprogrammer

2.5K followers · 25 following

Following ▾

I break Python & FastAPI apps in production... then fix them. Real bugs, performance issues, and solutions that actually work.

No responses yet



W Gong

What are your thoughts?

More from inprogrammer



 In Stackademic by inprogrammer · Jul 12

I Built a Tiny GPT From Scratch in Under 40 Lines of Python. Here's...

Every explainer video about GPT tells you the same thing: “it predicts the next token.” Fine...

★ 🖱️ 358 💬 7



 In AI Advances by Andrus · Jun 12

I Went Looking for the Best Local LLM for Coding in 2026.

In February 2025 I wrote a guide on running DeepSeek R1 locally, then quietly stopped...

★ 🖱️ 1.1K 💬 24 ↺ 12



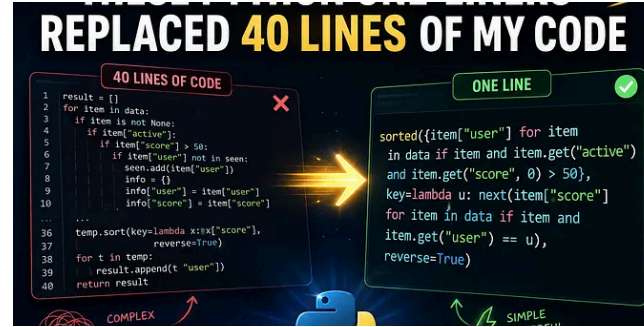


 In AI Advances by Jose Crespo, PhD · Jul 7

GTA 6 Revealed: When Pure Geometry Beats the Most...

Take-Two's Patents Reveal Some of the Game's Best-Kept Technical Secrets

★ 🖐️ 613 💬 11 ↺ 6



 In Stackademic by inprogrammer · Jun 9

These Python One-Liners Replaced 40 Lines of My Code. I'...

Three months ago, a senior engineer reviewed my pull request.

★ 🖐️ 282 💬 3



See all from inprogrammer

Recommended from Medium



In Towards AI by allglenn · Jul 19

How to Use Claude Code with Kimi K3 (and Switch Models by Typing...

A verified, step-by-step guide to running Claude Code against Moonshot AI's Kimi K3...

★ 🖱️ 132 💬 1 ↺ 1

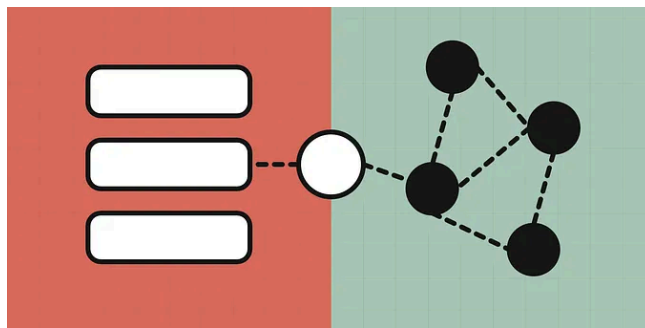


In The Code Frontier by Secret Dev · Jul 3

Beyond RAG: How Google's Open Knowledge Format (OKF) is...

For the last three years, the default engineering response to any enterprise AI...

★ 🖱️ 2K 💬 34 ↺ 24



Udaykiran Estari · Jul 3

Stop Wasting LLM Tokens: Building a Self-Updating Codebase...



In Artificial Intelligence in Plain E... by Sajit... · Jul 2

Meet Ornith: The Agentic Coding Model That Runs Entirely on Your...

Google's new Open Knowledge Format (OKF) standardizes context. Here's how to build a...

★ 🖱️ 405 💬 5 ↺ 2



★ 🖱️ 280 💬 7 ↺ 3



 In No Time by Pranit naik · Jul 19

10 Best Use Cases of Kimi K3 (The Model That Made US Stocks Fall...

It beats Claude while being open source. So let's see just how powerful it really is.

★ 🖱️ 647 💬 7 ↺ 4



 In Syntest by Shivam Bharadwaj · Jul 13

I Stopped Paying for AI — Here's the Free Local Setup That Replac...

No API Key. No Internet. No Bill. Running Gemma 4 on Your Laptop in 10 Minutes

★ 🖱️ 335 💬 2



See more recommendations

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Store](#) [Privacy](#) [Rules](#) [Terms](#) [Text to speech](#)